

INTRODUCCIÓ A GIT I GITHUB

Introducció

Què és?

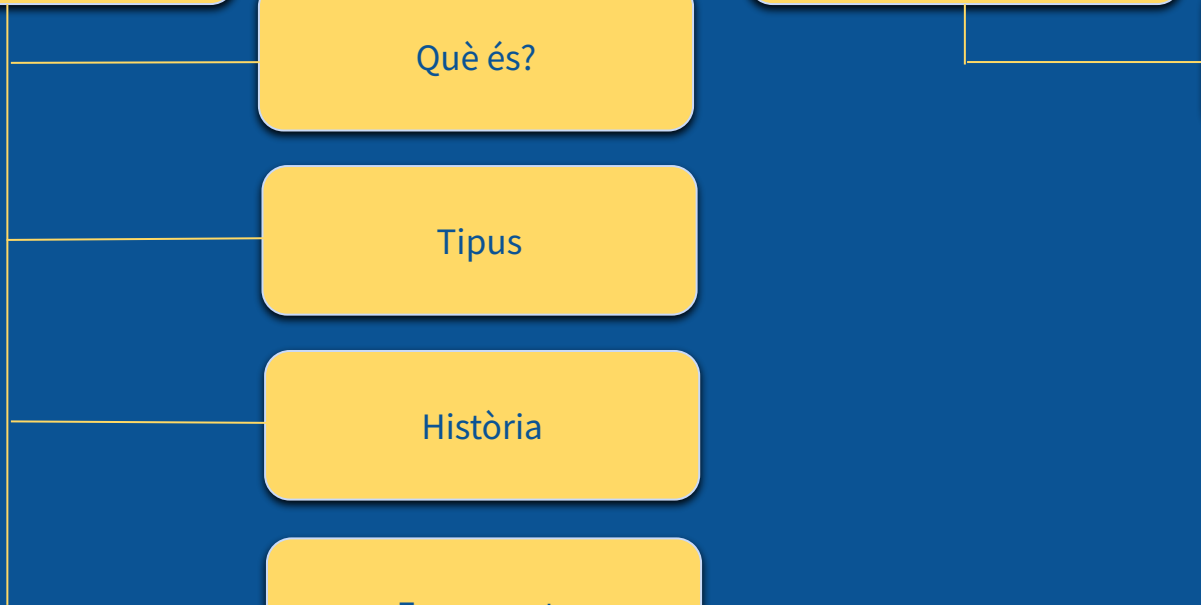
Tipus

Història

Fonaments

Començant amb Git

Fonaments de Git



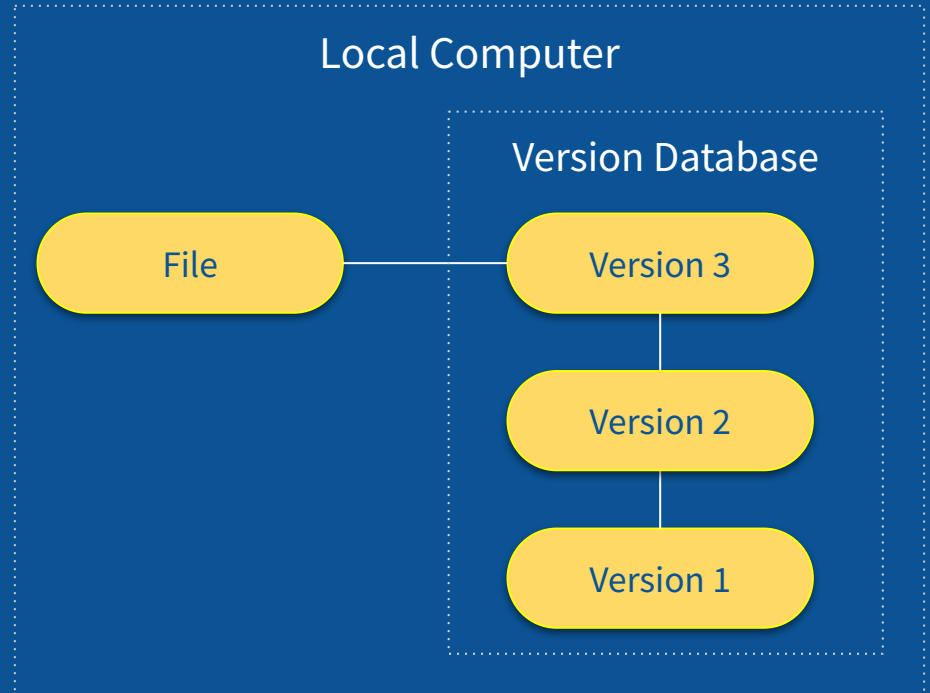
INTRODUCCIÓ

Autor: Roger Sobrino Gil

Què és?

- GIT és un control de versions (VCS - Version Control System).
- Sistema que registra els canvis realitzats a un o varis arxius.
- Facilitat en recuperació de versions anteriors dels arxius.

- Sistema de Control de Versions Locals
 - Registre dels canvis en una BBDD simple



Tipus - Centralitzats (CVCS)

- VCS Centralitzat

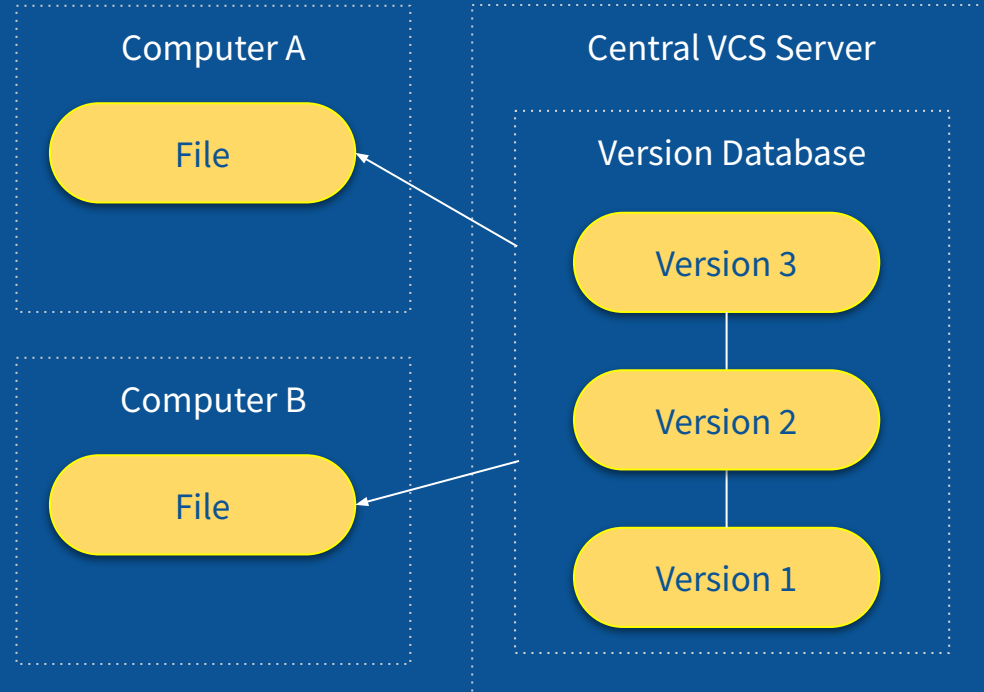
- Un servidor conté tots els arxius
- Clients descarreguen els arxius

Avantatge

- Coneixement de qui està fent què

Desavantatges

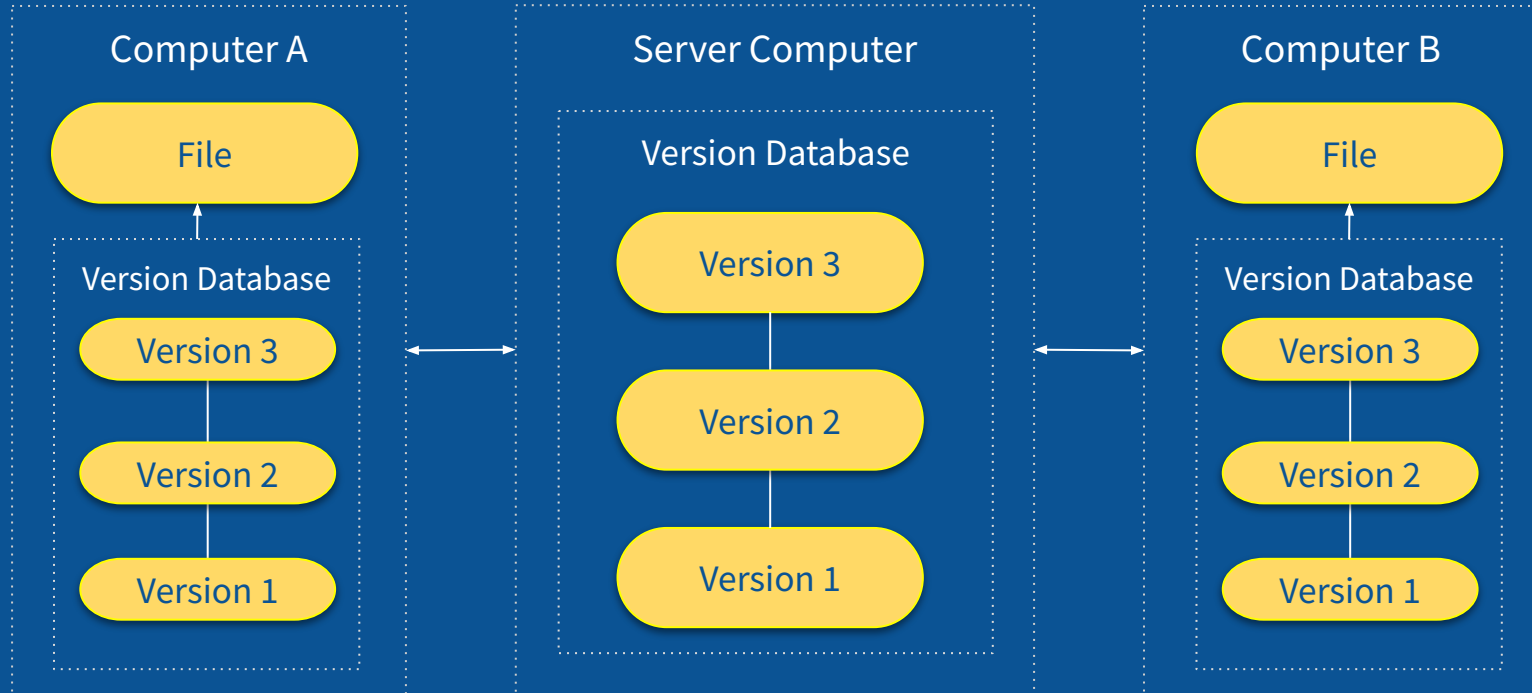
- Desconnexió del server implica no poder col·laborar ni guardar canvis.
- Pèrdua d'informació en cas que la BBDD estigui corrompuda



- Sistema de Control de Versions Distribuïdes
 - En la descàrrega es repliquen tots els arxius del repositori.
 - Col·laboració simultànea amb gran varietat de persones en el mateix projecte.
 - Varis fluxes de treball.
 - En caiguda del servidor, els repositoris dels clients poden ser copiats al servidor per restaurar-lo.

Introducció

Tipus - Distribuïdes (DVCS)

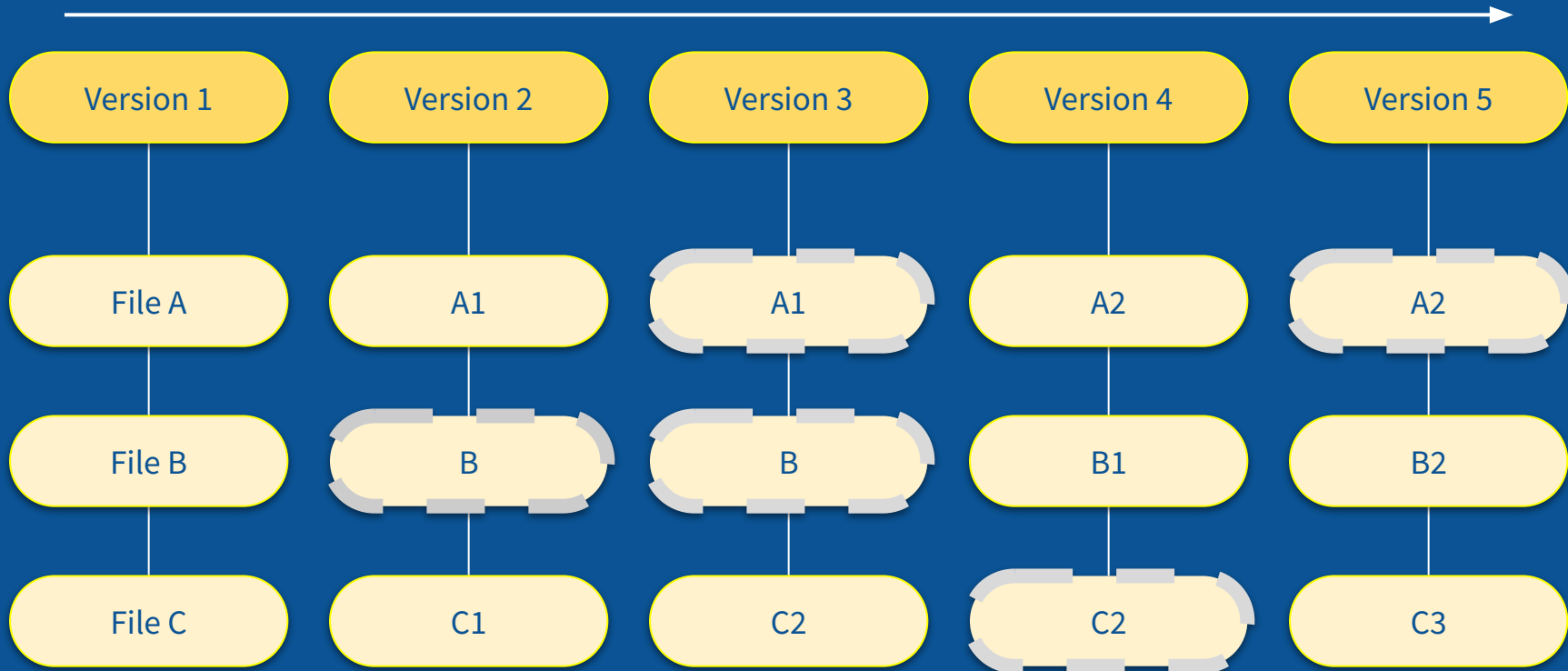


- Entre el 1991 i 2002 el manteniment del Kernel de Linux es feien amb parxes i arxius.
- 2002 el projecte del Kernel de linux van començar a utilitzar BiKeeper (DVCS).
- 2005 deixen d'utilitzar BiKeeper. Comunitat de Linux s'encarrega de desenvolupar el seu propi DVCS. → **Git**
 - Ràpid
 - Gran sistema de ramificació (Branching)
 - Molt eficient en grans projectes

Introducció

Fonaments - còpies instantànies, no diferenciades

Checkins Over Time



- Tracta les dades com còpies instantànies.
- Si l'arxiu no s'ha modificat, no emmagatema l'arxiu.

- Els arxius estan a la màquina local
 - Major velocitat de navegació entre arxius.
 - Operacions immediates.
 - Navegació de Git sense estar connectat al servidor.

- Git verifica mitjançant Checksum
 - Cada canvi de contingut es queda enregistrat.
 - Hash SHA-1
 - Git guarda tot, no per nom, sinó per número de hash.

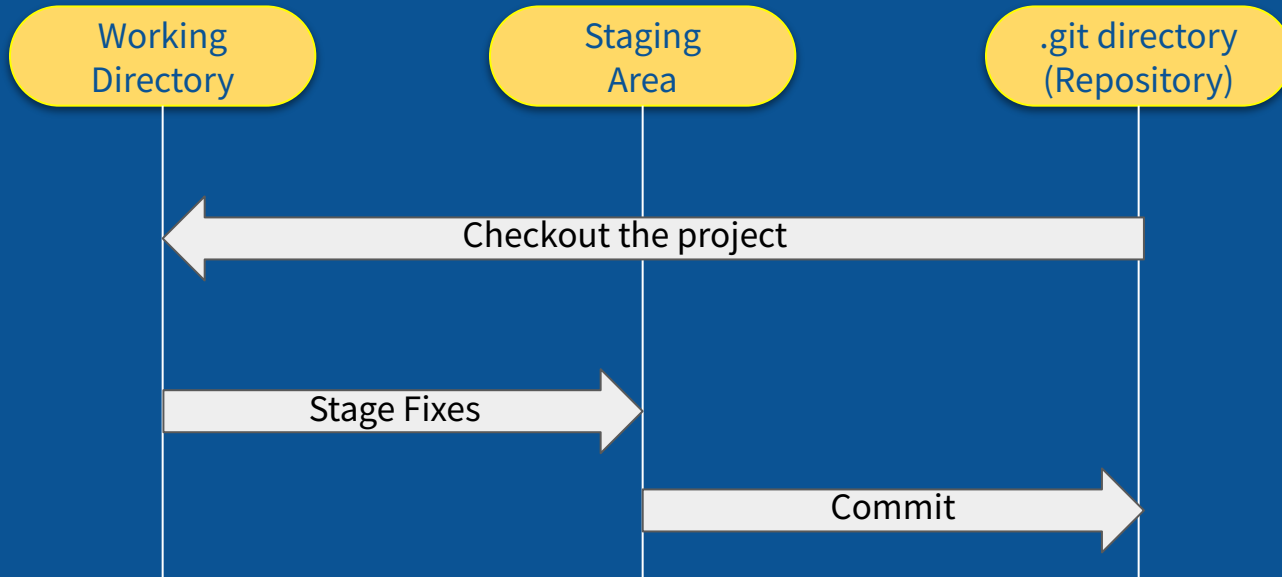
- Git té tres estats:
 - Confirmat (Committed)
Dades emmagatzemades de forma segura en la BBDD local.
 - Modificat (Modified)
L'arxiu està modificat però no confirmat en la BBDD local.
 - Preparat (staged)
S'ha marcat un arxiu modificat per a que vagi a la pròxima confirmació.

- Git té tres àrees de treball:
 - Directori de Git - **Git Directory (Repositori)**
Base de dades comprimida de tot el contingut passat i present
 - Directori de treball - **Work Directory**
Versió actual dels arxius/subdirs amb els que es treballa
 - Àrea de preparació - **Staging Area**
Arxiu contingut al directori de Git. Conté informació del que hi haurà a la pròxima confirmació.

- Flux de treball bàsic en Git:
 - Es modifica una serie d'arxius en el directori de treball.
 - Es preparen els arxius, afegint-los a l'àrea de preparació.
 - Es confirmen els canvis, s'agafen els arxius de l'àrea de preparació i emmagatzema la còpia de forma permanent al directori de Git.

Introducció

Fonaments - Tres sessions



Linux

Windows

- Git utilitza la linea de comandes (Terminal, Command Prompt o PowerShell)

Mac

- Instal·lació

- Linux

`sudo apt-get install git`

- Mac

<https://git-scm.com/download/mac>

- Windows

<https://git-scm.com/download/win>

COMENÇANT AMB GIT

CICLE FORMATIU DE GRAU SUPERIOR
DESENVOLUPAMENT D'APLICACIONS MULTIPLATAFORMA / WEB

En el directori del projecte existent i **NO existeix repositori a github...**

\$ git init - Crea un subdirectori amb nom .git. Aquest arxiu conté tots els arxius necessaris del repositori. Encara no es té cap arxiu del projecte sota seguiment.

\$ git add *.c

\$ git add LICENSE

\$ git commit -m 'initial project version' - Començar a fer el seguiment dels arxius del directori del projecte i fer una confirmació inicial amb el missatge 'initial project version'

Si vols col·laborar en un projecte existent...

\$ git clone https://github.com/roger_sobrino/primer_projecte1 - Es còpia el repositori existent a la direcció url del projecte a clonar.

Es crea una carpeta amb nom **primer_projecte1** i inicialitza un directori amb nom **.git**. Descarrega tota la informació del repositori i extreu una còpia de la última versió.

\$ git clone https://github.com/roger_sobrino/primer_projecte1 myFirstProject - Per clonar el repositori dintre d'una carpeta amb nom **myFirstProject**.

Guardar canvis en repositori

- Cada arxiu del repositori pot tindre 2 estats:
 - Rastrejats (Tracked files):

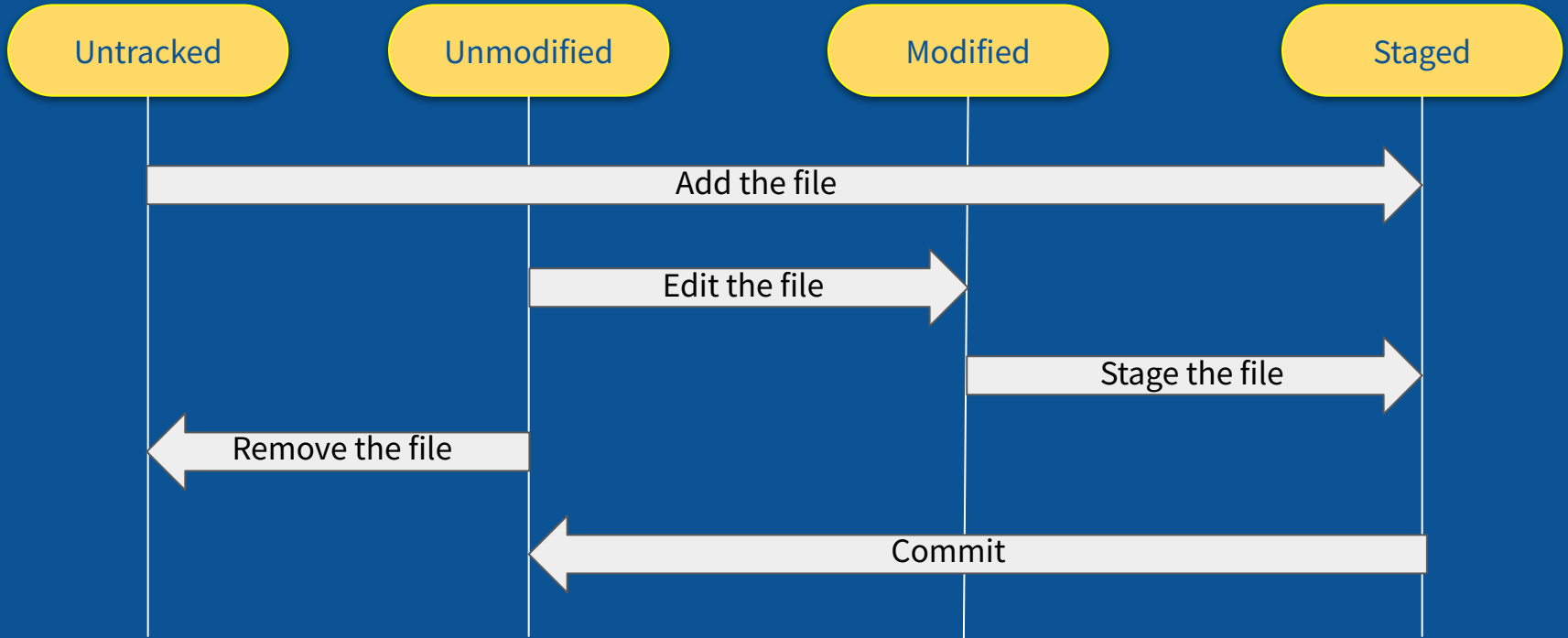
Son els arxius que estan a la última instantània del projecte (arxius modificats, sense modificar o preparats).
 - No rastrejats:

Qualsevol altre arxiu que no estava en la última instantània i que no està en l'àrea de preparació (staging area).

Quan clones per primera vegada un repositori, tots els arxius estaran rastrejats i sense modificar (ja que no han sigut editats encara).

Començant amb Git

Guardar canvis en repositori



Revisar l'estat dels arxius

\$ git status - Comanda per determinar l'estat dels arxius.

Si executes la comanda **\$ git status** immediatament després de clonar un repositori s'hauria de mostrar la següent informació:

\$ git status

On branch master

nothing to commit, working directory clean

Director de treball net.

No hi ha arxius modificats ni rastrejats.

“On branch master” indicar a quina branca estas

Revisar l'estat dels arxius

Es crea un arxiu amb nom README que conté el text **My Project**.

```
$ echo 'My Project' > README
```

```
$ git status
```

On branch master

Untracked files:

(use "git add <file>..." to include in what will be committed)

README

nothing added to commit but untracked files present (use "git add" to track)

Missatge que us avisa de què es pot fer

L'arxiu README està sense rastrejar (untracked file). Significa que Git veu arxius que no estaven en el commit anterior.

Revisar l'estat dels arxius

Comanda per afegir
arxius/directoris i que
es puguin rastrejar

```
$ git add README
$ git status
On branch master
Changes to be committed:
  (use "git reset HEAD <file>..." to unstage)

    new file:   README
```

Missatge que us avisa
de què es pot fer

L'arxiu README està rastrejat i llest per a ser confirmat

Començant amb Git

Preparar arxius modificats

Si es modifica l'arxiu amb nom **CONTRIBUTING.md** i s'utilitza la comanda **\$ git status** es veurà el missatge de la imatge.

\$ git status

On branch master

Changes to be committed:

(use "git reset HEAD <file>..." to unstage)

new file: README

Changes not staged for commit:

(use "git add <file>..." to update what will be committed)

(use "git checkout -- <file>..." to discard changes in working directory)

modified: CONTRIBUTING.md

Changes not staged for commit - Canvis no preparats per confirmar

Existeix un arxiu **rastrejat** que ha sigut **modificat** i que **no** està **preparat**

Començant amb Git

Preparar arxius modificats

```
$ git add CONTRIBUTING.md
```

```
$ git status
```

On branch master

Changes to be committed:

(use “git reset HEAD <file>...” to unstage)

new file: README

modified: CONTRIBUTING.md

Preparar l'arxiu CONTRIBUTING.md

Els arxius **README** i **CONTRIBUTING.md** estan preparats i formaran part de la pròxima confirmació.

Començant amb Git

Preparar arxius modificats

Si es fa algun canvi a l'arxiu **CONTRIBUTING.md** abans de confirmar-lo i utilitzes la comanda **\$ git status** es veurà el següent:

```
$ echo 'text de prova' > CONTRIBUTING.md
```

```
$ git status
```

On branch master

Changes to be committed:

(use "git reset HEAD <file>..." to unstage)

new file: README

modified: CONTRIBUTING.md

Changes not staged for commit:

(use "git add <file>..." to update what will be committed)

(use "git checkout -- <file>..." to discard changes in working directory)

modified: CONTRIBUTING.md

L'arxiu **CONTRIBUTING.md**, que és el mateix, apareix dues vegades (com preparat i no preparat).

Git prepara un arxiu segons l'estat en el que està quan s'executa la comanda **\$ git add**. En aquest cas, no s'ha utilitzat aquesta comanda i, per tant, es veuen les dues versions del mateix arxiu.

Preparar arxius modificats

```
$ git add CONTRIBUTING.md
```

```
$ git status
```

On branch master

Changes to be committed:

(use “git reset HEAD <file>...” to unstage)

new file: README

modified: CONTRIBUTING.md

A l'executar **\$ git add** es confirmen els canvis de **CONTRIBUTING.md** i es prepara la última versió del mateix.

Començant amb Git

Estat abreujat

```
$ git status -s
```

```
M README
```

```
MM Rakefile
```

```
A lib/git.rb
```

```
M lib/simplegit.rb
```

```
?? LICENSE.txt
```

La comanda **\$ git status -s** o **\$ git status --short** per obtenir informació de forma abreujada

?? - Arxius nous que no estan rastrejats.

A - Arxius que estan preparats.

M - Arxius que estan modificats.

MM - Arxius modificats que estan preparats i han sigut modificats altra vegada.

L'estat d'un arxiu apareix en dues columnes; la columna de l'esquerra indica l'estat preparat i el de la dreta indica l'estat sense preparar.

Exemple; l'arxiu **README** està modificat pero no està preparat ja que l'estat de l'esquerra el té en blanc i l'estat de la dreta té una **M**. En canvi, l'arxiu **lib/simplegit.rb** està modificat i preparat.

Ignorar arxius

```
$ cat .gitignore
```

```
*.[oa]
```

```
~
```

La comanda **\$ cat .gitignore** mostra un arxiu ocult de nom `.gitignore`. Aquest arxiu tindrà un llistat d'arxius que no es volen afegir automàticament a Git.

`*.[oa]` → Ignorar qualsevol arxiu que acabit per “a” o per “o”.

`~` → Ignorar tots els arxius que comencin per la tilde (~).

Crear un arxiu **.gitignore** és molt recomanable per evitar confirmacions accidentals d'arxius que no es volen incloure.

Patrons a incloure en **.gitignore**:

Ignorar les línies blanques i les que comencin per #.

Emplear patrons glob

Glob → expresions regulars com; [abc], [?], [0-9], [*]...

Per mirar una llista més extensa d'aquests patrons anar a [l'enllaç](#).

Exemple d'un arxiu **.gitignore**.

```
# ignora els arxius acabats en .a
```

```
*.a
```

```
# pero no lib.a, tot i ignorar els arxius acabats en .a demanat en la línia anterior
```

```
!lib.a
```

```
# ignora tots els arxius del directori build/
```

```
build/
```

```
# ignora doc/notes.txt, però no aquest: doc/server/arch.txt
```

```
doc/*.txt
```

```
# ignora tots els arxius .txt del directori doc/
```

```
doc/**/*.txt
```

Començant amb Git

Confirmar canvis

```
$ git commit -m "message"
```

```
[main de8123b] afegit message
```

```
1 file changed, 1 insertion(+), 4 deletions(-)
```

La comanda **\$ git commit -m "message"** serveix per confirmar els canvis de l'arxiu o arxius que estan en el estat **stage**. Els arxius que no estiguin al estat **stage** no seran confirmats.

En el mateix missatge de resposta del commit ens dona informació dels arxius adjunts i les modificacions.

Si no es posa **-m "message"** al commit, saltarà l'editor vim o emacs i haureu de posar el missatge manualment. Es recomana utilitzar l'opció del requadre negre.

Pujar canvis al servidor

\$ git push

Enumerating objects: 5, done.

Counting objects: 100% (5/5), done.

Delta compression using up to 8 threads

Compressing objects: 100% (3/3), done.

Writing objects: 100% (3/3), 305 bytes | 305.00 KiB/s,
done.

Total 3 (delta 1), reused 0 (delta 0), pack-reused 0

remote: Resolving deltas: 100% (1/1), completed with
1 local object.

To https://github.com/xxxxxxx/xx.git

05b032a..de8123b main -> main

La comanda **\$ git push** serveix per a pujar tots els arxius al servidor (github) una vegada fet el commit.

Exemple de missatge en resposta a la pujada al servidor dels nous arxius.

MOLTES GRÀCIES!

